

1)

- Web (HTTP): 80.
- SSH: 22 (Conocimiento externo e historial de conversación).
- DNS: 53.
- Web Seguro (HTTPS): 443.
- POP3: 110.
- IMAP: 143.
- SMTP: 25.

Ubicación de la asociación:

- En Linux, esta correspondencia de puertos y servicios se describe en el archivo `/etc/services`
- En Windows, esta asociación se encuentra en el archivo `C:\Windows\System32\drivers\etc\services`

2) El multicast es un tipo de direccionamiento en el que la información se destina a un grupo específico de hosts simultáneamente, en una o varias redes (esquema uno-a-muchos o muchos-a-muchos).

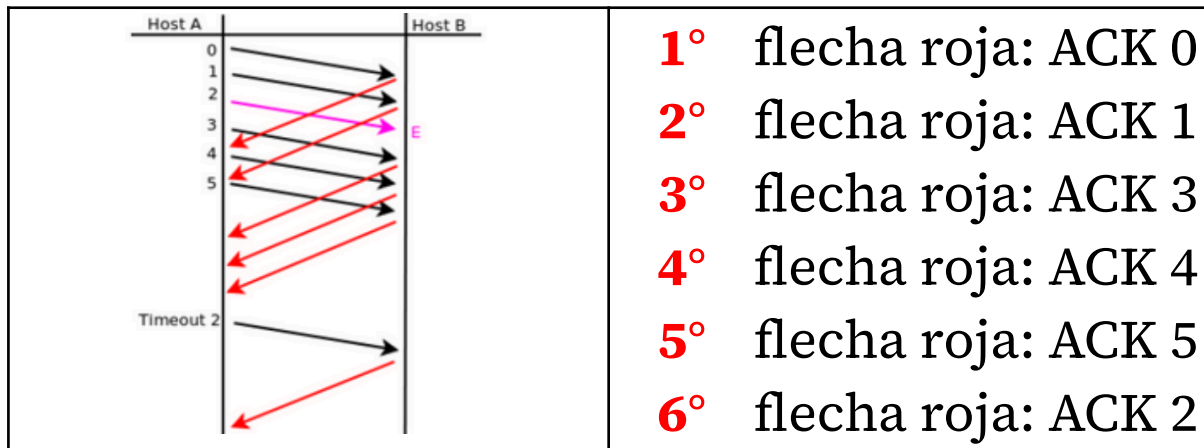
Funciona sobre el protocolo UDP.

No se podría adaptar para que funcione sobre TCP. TCP es un protocolo estrictamente orientado a la conexión y punto a punto (un único emisor y un único receptor). TCP requiere mantener el estado de la conexión (buffers, control de flujo, números de secuencia y acuses de recibo) para ese receptor individual. Adaptarlo a múltiples receptores simultáneos haría colapsar su mecanismo de confiabilidad y control de congestión, ya que no podría sincronizar los ACKs ni el tamaño de ventana de decenas o miles de destinos distintos.

3)

- Modo Activo: El cliente se conecta al puerto 21 del servidor para enviar comandos de control e indica mediante el comando PORT en qué puerto local escuchará los datos. Luego, el servidor inicia activamente la conexión de datos desde su puerto 20 hacia el puerto indicado por el cliente. Suele ser problemático con firewalls y NAT.
- Modo Pasivo: El cliente solicita este modo con el comando PASV. El servidor responde con un puerto no privilegiado (efímero) en el que se queda escuchando. Luego, el cliente inicia la conexión de datos hacia ese puerto del servidor.
- Diferencia principal con otros protocolos: A diferencia de HTTP o SMTP que envían comandos y datos mezclados sobre una misma conexión, FTP utiliza conexiones separadas (información de control fuera de banda o Out-Of-Band). Emplea una conexión persistente para comandos (puerto 21) y abre/cierra conexiones de datos dinámicas por cada transferencia de archivo.

4)



- 5) En el protocolo de Repetición Selectiva, el tamaño de la ventana debe ser estrictamente menor o igual a un límite respecto al espacio total de los números de secuencia. Si la ventana es demasiado grande (por ejemplo, ventana de tamaño 3 con números de secuencia 0, 1, 2, 3), el receptor no podrá distinguir si un paquete que llega es una retransmisión de un paquete antiguo perdido o un paquete completamente nuevo de la siguiente ventana. La regla matemática exacta dicta que el tamaño de la ventana (N) debe ser menor o igual a la mitad del espacio de los números de secuencia ($N \leq \text{Espacio de Secuencia}/2$).

6)

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	172.20.1.1	172.20.1.100	TCP	74	41749 > vce [SYN, Seq=1047471501, Win=5840, Len=0, MSS=1460, SACK_PERM=1, TSval=270132, TSecr=0]
2	0.001264	172.20.1.100	172.20.1.1	TCP	74	vce > 41749 [SYN, ACK] Seq=1047471501, Ack=3933822138, Win=5792, Len=0, MSS=1460, SACK_PERM=1, TSval=270132, TSecr=1877442
3	0.001341			TCP	66	> [ACK] Seq=1047471501, Ack=3933822138, Win=5888, Len=0, TSval=270132, TSecr=1877442

Internet Protocol Version 4, Src: 172.20.1.100 (172.20.1.100), Dst: 172.20.1.1 (172.20.1.1)

Transmission Control Protocol, Src Port: vce (11111), Dst Port: vce (41749), Seq: 1047471501, Ack: 3933822138, Len: 0

Source port: vce (11111)
 Destination port: 41749 (41749)
 [Stream index: 0]
 Sequence number: 1047471501
 Acknowledgement number: 3933822138
 Header length: 40 bytes

Flags: 0x012 (SYN, ACK)

000. = Reserved: Not set
 ...0 = Nonce: Not set
 0... = Congestion Window Reduced (CWR): Not set
0.. = ECN-Echo: Not set
0. = Urgent: Not set
1 = Acknowledgement: Set
 0.. = Push: Not set
0.. = Reset: Not set
1. = Syn: Set
0 = Fin: Not set
 Window size value: 5792
 [Calculated window size: 5792]
 Checksum: 0x9803 [validation disabled]

- 1er borroneo en paquete 1, columna Info: "SYN"
- 2do borroneo en el paquete 1, columna Info: "1047471501"
- Borroneo en paquete 3, columna Source: "172.20.1.1"
- Borroneo en paquete 3, columna Destination: "172.20.1.100"
- 1er borroneo en paquete 3, columna Info: "41749"
- 2do borroneo en paquete 3, columna Info: "vce"
- 3er borroneo en paquete 3, columna Info (entre corchetes): "ACK"
- 4to borroneo en paquete 3, columna Info: "3933822138" porque el número de secuencia del cliente en este paso es igual al número de acuse de recibo (Ack) que le envió el servidor en el paso anterior

- 5to borroneo en paquete 3, columna Info: “1047471502” porque el cliente confirma la recepción del paquete SYN, ACK del servidor sumándole 1 a su número de secuencia original (1047471501 + 1)

7)

Paquete	t	Acción	Seq	Ack	Explicación
3	1.360	Finaliza el saludo a tres vías	1	1	Seq: Usa el número que el otro host indicó en su Ack Ack: Confirma el SYN del host 10.0.1.10 (que tenía Seq=0, así que $0 + 1 = 1$)
5	3.581	Confirma el 1er bloque de datos	1	8	Ack: El paquete anterior envió 7 bytes (Seq=1 y Len=7). El receptor espera el próximo byte: $1 + 7 = 8$
7	8.797	Confirma el 2do bloque de datos	1	17	Ack: El paquete anterior envió 9 bytes (Seq=8 y Len=9). El receptor espera el próximo: $8 + 9 = 17$
9	14.382	Confirma el 3er bloque de datos	1	22	Ack: El paquete anterior envió 5 bytes (Seq=17 y Len=5). El receptor espera el próximo: $17 + 5 = 22$
10	15.190	El host 10.0.0.10 inicia el cierre de la conexión	22	1	Seq: Retoma desde el último Ack que le envió el receptor (el del paquete 9)
11	15.190	El host 10.0.1.10 confirma el FIN y envía el suyo	1	23	Ack: Confirma el FIN del paquete anterior. Como el flag FIN consume 1 número de secuencia, se le suma 1 al Seq recibido: $22 + 1 = 23$
12	15.190	El host 10.0.0.10 confirma el cierre final	23	2	Seq: Utiliza el número que el otro host le indicó que estaba esperando (Ack=23 en el paquete 11)

8) El RTT (Round Trip Time) es el tiempo de ida y vuelta; es decir, el tiempo que transcurre desde que un segmento es enviado hasta que se recibe su correspondiente acuse de recibo (ACK).

Cálculo: TCP mantiene un promedio suavizado mediante una media móvil exponencialmente ponderada, calculada como:

$$RTT_{Estimado} = (1-\alpha) \cdot RTT_{Estimado} + \alpha \cdot RTT_{Muestra} \text{ (típicamente } \alpha = 0,125 \text{).}$$

También se calcula una desviación del RTT (RTT_{Desv}).

TCP Timestamp, TSval y TSecr: Las marcas de tiempo (TCP timestamps) se introducen como extensiones (Opciones TCP) para medir el RTT de forma más precisa, especialmente en redes de alta velocidad.

El emisor coloca su marca de tiempo del reloj actual en el campo TSval (Timestamp Value). El receptor, al confirmar el segmento, copia ese mismo valor exacto en el campo TSecr (Timestamp Echo Reply). Así, al recibir el ACK,

el emisor resta el TSecr de su reloj actual para calcular un RTT exacto, solucionando la ambigüedad en las retransmisiones.

9)

a) Hay 6:

1. 3 0.000079 10.0.2.10 10.0.4.10 TCP 74 46907 → 5001 [SYN] Seq=0
Win=14600 Len=0 MSS=1460 SACK_PERM=1 TSval=120632 TSecr=0
WS=16
2. 961 82.420045 10.0.2.10 10.0.4.10 TCP 74 45670 → 7002 [SYN] Seq=0
Win=14600 Len=0 MSS=1460 SACK_PERM=1 TSval=141236 TSecr=0
WS=16
3. 963 83.540758 10.0.2.10 10.0.4.10 TCP 74 45671 → 7002 [SYN] Seq=0
Win=14600 Len=0 MSS=1460 SACK_PERM=1 TSval=141517 TSecr=0
WS=16
4. 967 97.968958 10.0.2.10 10.0.4.10 TCP 74 46910 → 5001 [SYN] Seq=0
Win=14600 Len=0 MSS=1460 SACK_PERM=1 TSval=145124 TSecr=0
WS=16
5. 981 135.753852 10.0.2.10 10.0.4.10 TCP 74 54424 → 9000 [SYN] Seq=0
Win=14600 Len=0 MSS=1460 SACK_PERM=1 TSval=154569 TSecr=0
WS=16
6. 1106 149.807117 10.0.2.10 10.0.4.10 TCP 74 54425 → 9000 [SYN]
Seq=0 Win=14600 Len=0 MSS=1460 SACK_PERM=1 TSval=158083
TSecr=0 WS=16

b)

1. 10.0.2.10:46937 → 10.0.4.10:5001
2. 10.0.2.10:45670 → 10.0.4.10:7002
3. 10.0.2.10:45671 → 10.0.4.10:7002
4. 10.0.2.10:46910 → 10.0.4.10:5001
5. 10.0.2.10:54424 → 10.0.4.10:9000
6. 10.0.2.10:54425 → 10.0.4.10:9000

c) Las conexiones exitosas se dan por el saludo a tres vías con los flags [SYN], [SYN, ACK], [ACK]. Hay 4:

1.

3 0.000079	10.0.2.10	10.0.4.10	TCP	74	46907 → 5001 [SYN] Seq=0 Win=14600 Len=0
MSS=1460 SACK_PERM=1 TSval=120632 TSecr=0 WS=16					
4 0.000116	10.0.4.10	10.0.2.10	TCP	74	5001 → 46907 [SYN, ACK] Seq=0 Ack=1
Win=14480 Len=0 MSS=1460 SACK_PERM=1 TSval=120650 TSecr=120632 WS=16					
5 0.151614	10.0.2.10	10.0.4.10	TCP	66	46907 → 5001 [ACK] Seq=1 Ack=1 Win=14608
Len=0 TSval=120669 TSecr=120650					

2.

967 97.968958	10.0.2.10	10.0.4.10	TCP	74	46910 → 5001 [SYN] Seq=0 Win=14600 Len=0
MSS=1460 SACK_PERM=1 TSval=145124 TSecr=0 WS=16					
968 97.969023	10.0.4.10	10.0.2.10	TCP	74	5001 → 46910 [SYN, ACK] Seq=0 Ack=1
Win=14480 Len=0 MSS=1460 SACK_PERM=1 TSval=145143 TSecr=145124 WS=16					
969 98.119589	10.0.2.10	10.0.4.10	TCP	66	46910 → 5001 [ACK] Seq=1 Ack=1 Win=14608
Len=0 TSval=145161 TSecr=145143					

3.

981 135.753852	10.0.2.10	10.0.4.10	TCP	74	54424 → 9000 [SYN] Seq=0 Win=14600 Len=0
MSS=1460 SACK_PERM=1 TSval=154569 TSecr=0 WS=16					
982 135.754058	10.0.4.10	10.0.2.10	TCP	74	9000 → 54424 [SYN, ACK] Seq=0 Ack=1
Win=14480 Len=0 MSS=1460 SACK_PERM=1 TSval=154588 TSecr=154569 WS=16					
983 135.904715	10.0.2.10	10.0.4.10	TCP	66	54424 → 9000 [ACK] Seq=1 Ack=1 Win=14608
Len=0 TSval=154608 TSecr=154588					

4.

```

1106 149.807117      10.0.2.10      10.0.4.10      TCP      74      54425 → 9000 [SYN] Seq=0 Win=14600 Len=0
MSS=1460 SACK_PERM=1 TSval=158083 TSecr=0 WS=16
1107 149.807136      10.0.4.10      10.0.2.10      TCP      74      9000 → 54425 [SYN, ACK] Seq=0 Ack=1
Win=14480 Len=0 MSS=1460 SACK_PERM=1 TSval=158102 TSecr=158083 WS=16
1108 149.957453      10.0.2.10      10.0.4.10      TCP      66      54425 → 9000 [ACK] Seq=1 Ack=1 Win=14608
Len=0 TSval=158121 TSecr=158102

```

d)

```

3 0.000079      10.0.2.10      10.0.4.10      TCP      74      46907 → 5001 [SYN] Seq=0 Win=14600 Len=0
MSS=1460 SACK_PERM=1 TSval=120632 TSecr=0 WS=16
4 0.000116      10.0.4.10      10.0.2.10      TCP      74      5001 → 46907 [SYN, ACK] Seq=0 Ack=1
Win=14480 Len=0 MSS=1460 SACK_PERM=1 TSval=120650 TSecr=120632 WS=16
5 0.151614      10.0.2.10      10.0.4.10      TCP      66      46907 → 5001 [ACK] Seq=1 Ack=1 Win=14608
Len=0 TSval=120669 TSecr=120650

```

- i) La conexión la inicia el host 10.0.2.10.
 - ii) El servidor es el host que recibe el paquete (10.0.4.10), y el cliente es quien lo envía primero (10.0.2.10).
 - iii) En los segmentos 3, 4 y 5.
 - iv) El paquete [SYN] tiene el nro de secuencia "2218428254". El paquete [SYN, ACK] tiene el nro de secuencia "1292618479". El paquete [ACK] tiene el nro de secuencia "2218428255".
 - v) MSS = 1460.
 - vi) .
- e) 6 0.151826 10.0.2.10 10.0.4.10 TCP 90 46907 → 5001 [PSH, ACK] Seq=1 Ack=1 Win=14608 Len=24 TSval=120670 TSecr=120650
- i) Lleva Len = 24 bytes de datos.
 - ii) Es confirmado por el segmento: 7 0.151925 10.0.4.10 10.0.2.10 TCP 66 5001 → 46907 [ACK] Seq=1 Ack=25 Win=14480 Len=0 TSval=120688 TSecr=120670.
 - iii) Confirma 24 bytes.
- f) 979 111.919605 10.0.2.10 10.0.4.10 TCP 66 46910 → 5001 [FIN, ACK] Seq=19 Ack=2 Win=14608 Len=0 TSval=148611 TSecr=148592

10)

- a) Lo activa el receptor. Lo hace enviando el tamaño del espacio libre en su buffer de recepción mediante el campo Ventana de recepción (RcvWindow o rwnd) en la cabecera del segmento TCP.
- b) Evita que un emisor rápido envíe demasiados datos seguidos y desborde (sature) la capacidad de almacenamiento del buffer de un receptor más lento.
- c) La restricción se mantiene activa mientras el campo RcvWindow sea cero o muy bajo. Se "desactiva" (se reanuda el flujo rápido) cuando el proceso de la aplicación en el receptor lee los datos, liberando espacio en el buffer, y el receptor notifica al emisor un nuevo valor de ventana de recepción mayor a cero.

11)

- a) Lo activa el emisor TCP. Los disparadores (sucesos de pérdida) son dos: el vencimiento de su temporizador (timeout) o la recepción de 3 ACKs duplicados.

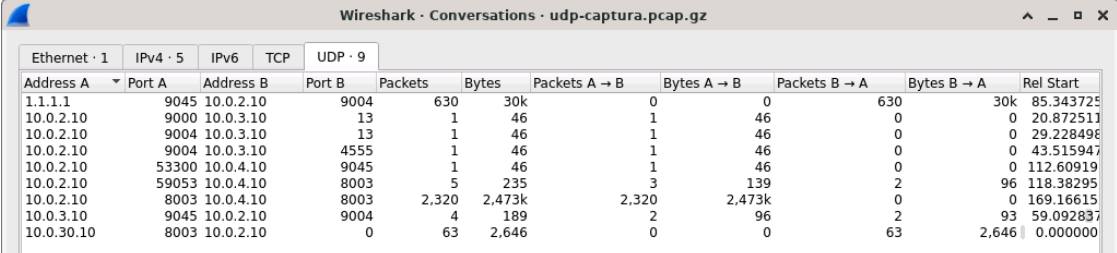
b) Evita el colapso de la red. Regula la velocidad del emisor para no inyectar demasiado tráfico, previniendo que se desborden los buffers de los enrutadores intermedios, lo que causaría pérdida masiva de paquetes y retardos graves.

c)

- **Slow Start:** Aumenta la tasa de transmisión de forma exponencial. La ventana de congestión crece 1 MSS por cada ACK recibido (duplicándose cada RTT). Se utiliza al iniciar la conexión o tras un timeout.
- **Congestion Avoidance:** Aumenta la tasa de transmisión de forma lineal. La ventana crece solo 1 MSS por cada RTT completo. Se ingresa a esta fase cuando la ventana de congestión alcanza el umbral de seguridad (ssthresh) para evitar congestionar la red súbitamente.

12)

a) Hay 9 comunicaciones



Address A	Port A	Address B	Port B	Packets	Bytes	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Rel Start
1.1.1.1	9045	10.0.2.10	9004	630	30k	0	0	630	30k	85.343725
10.0.2.10	9000	10.0.3.10	13	1	46	1	46	0	0	20.872511
10.0.2.10	9004	10.0.3.10	13	1	46	1	46	0	0	29.228496
10.0.2.10	9004	10.0.3.10	4555	1	46	1	46	0	0	43.515947
10.0.2.10	53300	10.0.4.10	9045	1	46	1	46	0	0	112.60919
10.0.2.10	59053	10.0.4.10	8003	5	235	3	139	2	96	118.38295
10.0.2.10	8003	10.0.4.10	8003	2,320	2,473k	2,320	2,473k	0	0	169.16615
10.0.3.10	9045	10.0.2.10	9004	4	189	2	96	2	93	59.092837
10.0.30.10	8003	10.0.2.10	0	63	2,646	0	0	63	2,646	0.000000

b)

- **Las exitosas:** Se ven cuando hay un intercambio fluido, es decir, un paquete de petición y su inmediata respuesta desde el otro extremo.
- **Las no exitosas:** A nivel red, se identifican con los mensajes de error ICMP. La captura muestra que se generaron mensajes ICMP de "Time-to-live exceeded", lo cual es el mecanismo estándar para informar que un datagrama UDP se descartó en el camino y no llegó a destino.

c) Sí, absolutamente. Aunque UDP no establece una conexión formal previa, se adapta perfectamente al modelo cliente/servidor. Un cliente envía un datagrama de solicitud a la dirección IP y puerto conocido de un servidor, y el servidor procesa la solicitud y envía un datagrama de respuesta de vuelta al cliente. Ejemplos clásicos son DNS o DHCP.

d)

- **Aplicaciones típicas:** DNS (resolución de nombres), DHCP (asignación de IPs), VoIP (telefonía IP), streaming de audio/video en vivo, y juegos multijugador online.
- **Requerimientos:** Estas aplicaciones priorizan la **velocidad y la baja latencia** por encima de la confiabilidad. Es preferible perder

un cuadro de un video o un milisegundo de audio (que el usuario apenas notará) antes que pausar toda la transmisión para esperar a que se retransmita un paquete perdido, lo cual arruinaría la experiencia en tiempo real.

- e) Hace lo mínimo indispensable. UDP incluye un campo de **Checksum (Suma de comprobación)** en su cabecera.
- Si el datagrama llega al destino y el cálculo del checksum coincide, asume que los datos están íntegros.
 - Si el checksum **falla** (los datos se corrompieron en el camino), el receptor simplemente **descarta y destruye el paquete en silencio**. UDP no notifica al emisor del error ni solicita ninguna retransmisión. No tiene mecanismos de acuse de recibo (ACKs) ni números de secuencia.
- f) En UDP observo comunicaciones muy cortas (a menudo de solo 1 paquete de ida y 1 de vuelta) a puertos específicos como el 53 (DNS). Además, en UDP es mucho más común ver direcciones IP de destino que terminan en .255 (tráfico de Broadcast) o en rangos de Multicast (ej. 224.0.0.x), usando puertos iguales tanto en origen como en destino (ej. protocolos de descubrimiento o anuncio local). Además, en TCP, los puertos efímeros del cliente se mantienen "anclados" al puerto del servidor durante toda la sesión.
- g) El primer datagrama es: 79 59.092837 10.0.2.10 10.0.3.10 UDP 46 9004 → 9045 Len=4.
- i)
- IP que envía: 10.0.2.10
 - Puerto de origen: 9004 (va hacia el puerto destino 9045).
- ii)
- Sentido de ida (Paquete 79): Se envían 4 bytes de datos útiles (Len=4).
 - Sentido de vuelta (Paquete 80, que es la respuesta de la IP 10.0.3.10): Se envían 7 bytes de datos útiles (Len=7).

13)

- a) `hping3 -p 3306 -udp 10.100.25.135`: envía paquetes UDP al puerto 3306. En la tabla, el puerto 3306 está a la escucha (LISTEN), pero bajo el protocolo TCP (servicio mysqld), no UDP. No hay ningún servicio escuchando tráfico UDP en ese puerto. **Devolverá un mensaje de error ICMP Port Unreachable (Destino Inalcanzable, Código 3: Puerto inalcanzable), ya que el puerto está cerrado para el protocolo UDP.**
- b) `hping3 -S -p 25 10.100.25.135`: realiza un escaneo TCP SYN (-S) al puerto 25. En la tabla, el puerto 25 (servicio master) está en estado LISTEN, pero su dirección local es 127.0.0.1:25. Esto significa que solo acepta conexiones desde la propia máquina (localhost). Como la consulta viene

desde una IP externa (10.100.25.90), el puerto aparecerá como cerrado.

Devolverá un paquete TCP con los flags RST, ACK (flags=RA), indicando que el puerto está cerrado o rechazando la conexión desde esa interfaz.

- c) `hping3 -S -p 22 10.100.25.135`: realiza un escaneo TCP SYN (-S) al puerto 22. El puerto 22 (servicio sshd) está en estado LISTEN en *:22. El asterisco indica que el servicio está escuchando en todas las interfaces de red de la máquina, incluyendo la interfaz que tiene la IP 10.100.25.135. **Al estar abierto, la máquina destino responderá con un paquete TCP con los flags SYN, ACK (flags=SA), aceptando el inicio del saludo a tres vías (Three-way handshake).**
- d) `hping3 -S -p 110 10.100.25.135`: realiza un escaneo TCP SYN (-S) al puerto 110. Si revisamos la lista de puertos en la columna Local Address:Port, el puerto 110 no figura en estado LISTEN en ninguna interfaz. Por lo tanto, el puerto está cerrado. **Al igual que en el caso 'b', devolverá un paquete TCP con los flags RST, ACK (flags=RA).**
- Hay 3 conexiones distintas establecidas. Si bien en la tabla hay 5 filas con el estado ESTAB (Established), 4 de ellas corresponden a tráfico interno de la máquina (localhost, 127.0.0.1). Cuando una conexión se realiza en localhost, la herramienta de red (como ss o netstat) registra ambos extremos de la misma conexión en líneas separadas (una vista desde el lado del cliente y otra desde el lado del servidor).
 - **Conexión 1 (Local):** Entre el puerto 3306 (mysqld) y el puerto efímero 34338 (ruby). Aparece dos veces en la tabla:
 - 127.0.0.1:3306 <-> 127.0.0.1:34338
 - 127.0.0.1:34338 <-> 127.0.0.1:3306
 - **Conexión 2 (Local):** Entre el puerto 3306 (mysqld) y el puerto efímero 48717 (ruby). También aparece dos veces:
 - 127.0.0.1:48717 <-> 127.0.0.1:3306
 - 127.0.0.1:3306 <-> 127.0.0.1:48717
 - **Conexión 3 (Externa):** Una conexión SSH activa desde una IP externa hacia la máquina. Solo se ve el lado del servidor local:
 - 10.100.25.135:22 <-> 200.100.120.210:61576